

Machine Learning - Python Control y Funciones

July 6, 2026

1 PA026-26 - Python



1.1 Estructuras de Control y Funciones

Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar

Este notebook combina el contenido de dos materiales de clase: 1. PA025_25_03_Python_Control (Estructuras de control) 2. PA025_25_04_Python_FUncion (Funciones)

- Repositorio: GitHub

2 Parte 1: Estructuras de Control

2.1 1. Selección (sentencias condicionales)

- Selección de una de varias alternativas en base a alguna condición.
- Indentación para estructurar el código.
- Importante el símbolo :

```
[5]: x = int(input("Ingrese el valor de x"))
# Implemento una condicional para saber si x es mayor a 0
if x > 0:
    print('Valor positivo')
else:
    print('Valor no positivo')

# Explicación línea por línea:
```

```
# 1) Se pide al usuario un número entero por teclado y se guarda en 'x'.
# 2) 'if x > 0' evalúa si x es estrictamente mayor que 0.
# 3) Si es verdadero, se imprime 'Valor positivo'.
# 4) 'else' cubre cualquier otro caso (x == 0 o x < 0), imprimiendo 'Valor no
    ↳positivo'.
# Resultado esperado (si el usuario ingresa -6): "Valor no positivo"
# Resultado esperado (si el usuario ingresa 6): "Valor positivo"
```

Ingrese el valor de x 45

Valor positivo

- Las condiciones son expresiones que se evalúan a True o False.
- Operadores de comparación, lógicos, de identidad y de pertenencia.

```
[6]: x = 3
y = [1, 2, 3]
print(x > 0)
print(x > 0 and x < 10)
print(x is not y)
print(x in y)

# Explicación línea por línea:
# 1) x = 3 (entero), y = [1,2,3] (lista).
# 2) x > 0 -> 3 > 0 es True.
# 3) x > 0 and x < 10 -> True and True -> True.
# 4) x is not y -> compara identidad de objetos; x (int) nunca es el mismo
    ↳objeto que y (list) -> True.
# 5) x in y -> comprueba si el valor 3 pertenece a la lista [1,2,3] -> True.
# Resultado esperado:
# True
# True
# True
# True
```

True

True

True

True

- Se pueden introducir más ‘ramas’ en sentencias condicionales a través de la palabra clave elif.

```
[7]: x = -3
if x > 0:
    print('Valor positivo')
elif x == 0:
    print('Valor nulo')
else:
```

```
print('Valor negativo')
```

```
# Explicación línea por línea:  
# 1) x = -3.  
# 2) if x > 0 -> False (no entra).  
# 3) elif x == 0 -> False (no entra).  
# 4) else -> se ejecuta porque ninguna condición anterior se cumplió.  
# Resultado esperado: "Valor negativo"
```

Valor negativo

- Anidamiento de condicionales (if dentro de if).

```
[8]: val = 120  
if val > 0:  
    if val < 100:  
        print('Valor positivo')  
    else:  
        print('Valor muy positivo')  
else:  
    print('Valor negativo')  
  
# Explicación línea por línea:  
# 1) val = 120.  
# 2) if val > 0 -> True, entra al bloque anidado.  
# 3) if val < 100 -> False (120 no es menor que 100).  
# 4) else (del if interno) -> se ejecuta, imprime 'Valor muy positivo'.  
# Resultado esperado: "Valor muy positivo"
```

Valor muy positivo

2.2 2. Switch (match-case)

```
[9]: a = -6  
match a:  
    case _ if a > 0:  
        print('positive')  
    case _ if a == 0:  
        print('zero')  
    case _ if a < 0:  
        print('negative')  
  
# Explicación línea por línea:  
# 1) a = -6.  
# 2) 'match a' evalúa 'a' contra distintos 'case'.  
# 3) Cada 'case _ if <condicion>' es un patrón comodín ('_') con guardia ↵  
    ↵ (condición extra).  
# 4) Se prueban en orden: a>0 (False), a==0 (False), a<0 (True) -> coincide.
```

```
# 5) Se imprime 'negative'.
# Resultado esperado: "negative"
```

negative

```
[10]: a = 'l' # week day
match a:
    case 'L':
        print("Lunes")
    case 'M':
        print("Martes")
    case 'X':
        print("Miercoles")
    case _:
        print("Wrong Day")

# Explicación línea por línea:
# 1) a = 'l' (minúscula).
# 2) match compara el valor exacto de 'a' contra los patrones literales 'L',
    ↪ 'M', 'X'.
# 3) Como la comparación es sensible a mayúsculas/minúsculas, 'l' no coincide
    ↪ con 'L'.
# 4) Se cae en el caso comodín 'case _', que imprime "Wrong Day".
# Resultado esperado: "Wrong Day"
# Nota: si a = 'L' (mayúscula), el resultado esperado sería "Lunes".
```

Wrong Day

2.3 Expresión ternaria

Estructura if-else condensada en una línea. Se recomienda usar solo en casos sencillos.

```
[11]: x = -3
resultado = x if x >= 0 else -x
print(resultado)

# Explicación línea por línea:
# 1) x = -3.
# 2) 'resultado = x if x >= 0 else -x' es una expresión ternaria:
#     si (x >= 0) es True, resultado = x; si es False, resultado = -x.
# 3) Como x = -3 < 0, resultado = -(-3) = 3. Esto calcula el valor absoluto de
    ↪ x.
# 4) print(resultado) imprime 3.
# Resultado esperado: 3
```

3

```
[12]: val = -3
if val >= 0:
```

```

    resultado = val
else:
    resultado = -val
print(resultado)

# Explicación línea por línea:
# 1) val = -3.
# 2) if val >= 0 -> False.
# 3) else -> resultado = -val = -(-3) = 3.
# 4) print(resultado) imprime 3. Es la versión equivalente sin ternario del
    ↪ejemplo anterior.
# Resultado esperado: 3

```

3

2.4 Concatenación de comparaciones

- and: true si ambos son true.
- or: true si al menos uno es true.
- not: invierte el valor de verdad.

```

[13]: genero = 'Drama'
fecha_de_estreno = 1989
if genero == 'Comedia' or genero == 'Acción':
    print('Buena película!')
elif fecha_de_estreno >= 1990 and fecha_de_estreno < 2000:
    print('Buena década!')
else:
    print('Meh')

# Explicación línea por línea:
# 1) genero = 'Drama', fecha_de_estreno = 1989.
# 2) if genero == 'Comedia' or genero == 'Acción' -> False or False -> False.
# 3) elif fecha_de_estreno >= 1990 and fecha_de_estreno < 2000 -> False (1989 <
    ↪1990) -> False.
# 4) else -> se ejecuta, imprime 'Meh'.
# Resultado esperado: "Meh"

```

Meh

2.5 Comparación de variables: == vs is

- == compara si el valor de las variables es el mismo.
- is compara si los objetos en las variables son iguales (misma referencia en memoria).

```

[14]: a = 1000
b = a
print(a is b)
print(a == b)

```

```

c = type(a)
print(c)
d = type(b)
print(d)
print(c == d)
print(id(a))
print(id(b))

# Explicación línea por línea:
# 1) a = 1000 (entero).
# 2) b = a -> b referencia al mismo objeto que a.
# 3) a is b -> como b apunta al mismo objeto, debería ser True (mismo id).
# 4) a == b -> compara valores, True.
# 5) c = type(a) -> <class 'int'>.
# 6) d = type(b) -> <class 'int'>.
# 7) c == d -> True, mismo tipo.
# 8-9) id(a) e id(b) muestran la dirección de memoria; al ser b=a deberían
    ↪ coincidir.
# Resultado esperado (en un intérprete estándar):
# True
# True
# <class 'int'>
# <class 'int'>
# True
# <mismo id para a y b, por ejemplo 140234567891232>
# <mismo id para a y b>
# Nota: el PDF original muestra un resultado distinto porque en esa ejecución
# se sobrescribió 'a' con un float en otra celda previa; en una ejecución
    ↪ limpia
# como esta, a y b son el mismo entero y todo debería dar True.

```

```

True
True
<class 'int'>
<class 'int'>
True
2725121343056
2725121343056

```

```

[15]: a = [1, 2, 3]
      b = [1, 2, 3]
      c = a
      print(a is b)
      print(a is c)
      print(a == b)
      print(id(a))
      print(id(b))

```

```

print(id(c))

# Explicación línea por línea:
# 1) a = [1,2,3] crea una lista nueva en memoria.
# 2) b = [1,2,3] crea OTRA lista distinta con el mismo contenido.
# 3) c = a -> c referencia al MISMO objeto que a.
# 4) a is b -> False, son objetos distintos aunque con igual contenido.
# 5) a is c -> True, mismo objeto.
# 6) a == b -> True, mismo contenido (comparación de valor, no de identidad).
# 7-9) id(a) e id(c) coinciden; id(b) es distinto.
# Resultado esperado:
# False
# True
# True
# <id_a>
# <id_b distinto de id_a>
# <id_a> (igual que id(a))

```

False

True

True

2725123281792

2725123162816

2725123281792

2.6 Valor booleano

- Todo objeto en Python tiene un valor booleano inherente: True o False.
- Cualquier número distinto de 0 o cualquier objeto no vacío tiene valor True.
- El número 0, objetos vacíos y None tienen valor False.

```

[16]: a = -10 # True
      b = None # False
      if a:
          print("a es True")
      if not b:
          print("b es False")

# Explicación línea por línea:
# 1) a = -10 -> cualquier número distinto de 0 se evalúa como True.
# 2) b = None -> None se evalúa como False.
# 3) if a -> como a es "truthy" (distinto de 0), se imprime "a es True".
# 4) if not b -> not False es True, por lo que se imprime "b es False".
# Resultado esperado:
# a es True
# b es False

```

a es True

b es False

```
[17]: c = [1, 2, 3]
# a c le reasigno None
c = None # None se evalúa como False, igual que una lista vacía
if c:
    print("Lista no vacía")
else:
    print("Lista vacía")

# Explicación línea por línea:
# 1) c = [1,2,3] (lista con elementos).
# 2) c se reasigna a None.
# 3) if c -> None es "falsy", por lo tanto la condición es False.
# 4) else -> se ejecuta, imprime "Lista vacía".
# Resultado esperado: "Lista vacía"
```

Lista vacía

2.7 3. Iteración (bucles)

Dos tipos: while y for.

2.7.1 Bucle while

Repite un bloque de código mientras se cumpla una condición.

```
[18]: indice = 0
numeros = [9, 4, 7, 1, 2]
while indice < 3:
    print(numeros[indice])
    indice = indice + 1

# Explicación línea por línea:
# 1) indice = 0 (contador inicial).
# 2) numeros = [9,4,7,1,2].
# 3) while indice < 3 -> se repite mientras indice sea menor que 3 (3_
    ↪ iteraciones: 0,1,2).
# 4) print(numeros[indice]) imprime el elemento en la posición actual.
# 5) indice = indice + 1 incrementa el contador para evitar bucle infinito.
# Resultado esperado:
# 9
# 4
# 7
```

9
4
7

```
[19]: numeros = [33, 3, 9, 21, 1, 7, 12, 10, 8]
contador = 0
```



```

indice = 0
while indice < len(numeros):
    if numeros[indice] < 10:
        print(numeros[indice])
        contador += 1
    indice += 1
print('Contador:', contador)
print('Indice:', indice)

# Explicación línea por línea:
# 1) numeros: lista de 9 elementos. contador e indice inician en 0.
# 2) while indice < len(numeros) -> recorre todas las posiciones (0 a 8).
# 3) if numeros[indice] < 10 -> filtra los números menores a 10: 3, 9, 1, 7, 8.
# 4) Cada vez que se cumple, se imprime el número y se incrementa 'contador'.
# 5) indice += 1 avanza siempre, esté o no dentro del if.
# 6) Al final se imprime cuántos números cumplieron la condición (contador) y
#     el valor final de indice (longitud de la lista, 9).
# Resultado esperado:
# 3
# 9
# 1
# 7
# 8
# Contador: 5
# Indice: 9

```

```

3
9
1
7
8
Contador: 5
Indice: 9

```

```

[20]: nombre = 'Pablo'
while nombre: # Mientras 'nombre' no sea vacío
    print(nombre)
    nombre = nombre[1:]

# Explicación línea por línea:
# 1) nombre = 'Pablo'.
# 2) while nombre -> un string no vacío es "truthy"; el bucle continúa mientras
    ↳ queden caracteres.
# 3) print(nombre) imprime el string actual.
# 4) nombre = nombre[1:] recorta el primer carácter en cada iteración (slicing).
# 5) Cuando nombre queda como '' (vacío), la condición del while se vuelve
    ↳ False y termina.

```

```
# Resultado esperado:  
# Pablo  
# ablo  
# blo  
# lo  
# o
```

Pablo
ablo
blo
lo
o

```
[21]: i = 0  
while i < 10:  
    print(i)  
    i += 1  
  
# Explicación línea por línea:  
# 1) i = 0.  
# 2) while i < 10 -> se repite mientras i sea menor que 10.  
# 3) print(i) imprime el valor actual.  
# 4) i += 1 incrementa i en 1 cada vuelta, garantizando que el bucle termine_  
    ↪(no es infinito).  
# Resultado esperado: los números del 0 al 9, cada uno en su propia línea.
```

0
1
2
3
4
5
6
7
8
9

2.7.2 Bucle for

Permite recorrer los items de una secuencia o un objeto iterable (strings, listas, tuplas, etc.).

```
[22]: peliculas = ['Matrix', 'The purge', 'Avatar', 'Star Wars']  
for pelicula in peliculas:  
    print(pelicula)  
  
# Explicación línea por línea:  
# 1) peliculas: lista de 4 strings.
```

```
# 2) for pelicula in peliculas -> en cada iteración 'pelicula' toma el valor de
    ↪ un elemento de la lista.
# 3) print(pelicula) imprime cada título en orden.
# Resultado esperado:
# Matrix
# The purge
# Avatar
# Star Wars
```

Matrix
The purge
Avatar
Star Wars

```
[23]: for i in range(10):
        print(i)

# Explicación línea por línea:
# 1) range(10) genera la secuencia 0,1,2,...,9 (10 valores, sin incluir el 10).
# 2) for i in range(10) -> 'i' toma cada uno de esos valores en orden.
# 3) print(i) imprime el valor actual de i.
# Resultado esperado: los números del 0 al 9, cada uno en su propia línea.
```

0
1
2
3
4
5
6
7
8
9

```
[24]: nombre = 'Pablo'
        for i in range(len(nombre)):
            print(nombre[i])

# Explicación línea por línea:
# 1) nombre = 'Pablo' (5 caracteres).
# 2) len(nombre) = 5, por lo que range(len(nombre)) genera 0,1,2,3,4.
# 3) for i in range(...) recorre esos índices.
# 4) nombre[i] accede al carácter en la posición i.
# Resultado esperado:
# P
# a
# b
# l
```

```
# 0
```

P
a
b
l
o

```
[25]: for letra in nombre:
      print(letra)

# Explicación línea por línea:
# 1) 'nombre' sigue siendo 'Pablo' de la celda anterior.
# 2) for letra in nombre -> itera directamente sobre los caracteres del string,
    ↪ sin necesidad de índices.
# 3) print(letra) imprime cada carácter.
# Resultado esperado:
# P
# a
# b
# l
# o
```

P
a
b
l
o

```
[26]: tuplas = [(1, 2, 4), (3, 4), (5, 6)]
      for a, b, c in tuplas:
          print(a, b, c)

# Explicación línea por línea:
# 1) tuplas: lista con tuplas de distinto tamaño (una de 3 elementos, dos de 2
    ↪ elementos).
# 2) for a, b, c in tuplas -> intenta desempaquetar cada tupla en exactamente 3
    ↪ variables (a, b, c).
# 3) La primera tupla (1,2,4) se desempaqueta correctamente: a=1, b=2, c=4 ->
    ↪ se imprime "1 2 4".
# 4) La segunda tupla (3,4) SOLO tiene 2 elementos, por lo que Python no puede
    ↪ asignarlos
#     a 3 variables y lanza: ValueError: not enough values to unpack (expected
    ↪ 3, got 2).
# Resultado esperado:
# 1 2 4
# ValueError: not enough values to unpack (expected 3, got 2)
```

1 2 4

```
-----
ValueError                                Traceback (most recent call last)
Cell In[26], line 2
      1 tuplas = [(1, 2, 4), (3, 4), (5, 6)]
----> 2 for a, b, c in tuplas:
      3     print(a, b, c)
      4
      5 # Explicación línea por línea:

ValueError: not enough values to unpack (expected 3, got 2)
```

```
[27]: diccionario = {'a': 1, 'b': 2, 'c': 3}
for key in diccionario:
    print(f"{key} => {diccionario[key]}")

# Explicación línea por línea:
# 1) diccionario: 3 pares clave-valor.
# 2) for key in diccionario -> al iterar un diccionario directamente, se
    ↪ recorren sus CLAVES.
# 3) diccionario[key] accede al valor asociado a esa clave.
# 4) El f-string arma la salida "clave => valor".
# Resultado esperado:
# a => 1
# b => 2
# c => 3
```

```
a => 1
b => 2
c => 3
```

```
[28]: for key, value in diccionario.items():
    print(f"{key} => {value}")

# Explicación línea por línea:
# 1) diccionario.items() devuelve pares (clave, valor) como tuplas.
# 2) for key, value in ... desempaqueta cada tupla directamente en dos
    ↪ variables.
# 3) Se imprime cada par con el mismo formato que antes, pero sin necesitar
    ↪ diccionario[key].
# Resultado esperado:
# a => 1
# b => 2
# c => 3
```

```
a => 1
```

```
b => 2
c => 3
```

```
[29]: for value in diccionario.values():
        print(value)

# Explicación línea por línea:
# 1) diccionario.values() devuelve solo los valores del diccionario, sin las
    ↪ claves.
# 2) for value in ... itera sobre esos valores.
# 3) print(value) imprime cada valor.
# Resultado esperado:
# 1
# 2
# 3
```

```
1
2
3
```

```
[30]: for a, b, c in [(1, 2, 3), (4, 5, 6)]:
        print(a, b, c)

# Explicación línea por línea:
# 1) Se itera directamente sobre una lista literal de tuplas de 3 elementos
    ↪ cada una.
# 2) a, b, c se desempaquetan de cada tupla en cada iteración.
# 3) print(a, b, c) imprime los tres valores separados por espacio.
# Resultado esperado:
# 1 2 3
# 4 5 6
```

```
1 2 3
4 5 6
```

2.8 3.1 Las sentencias break, continue y else

- break: termina el bucle por completo.
- continue: salta a la siguiente iteración.

```
[31]: rating_to_find = 4.2
movie_ratings = [4.3, 2.5, 1.7, 4.2, 3.8, 3.3, 4.5]
for rating in movie_ratings:
    print(rating)
    if rating == rating_to_find:
        print("Found")
        break

# Explicación línea por línea:
```

```

# 1) rating_to_find = 4.2 es el valor buscado.
# 2) for rating in movie_ratings recorre la lista en orden.
# 3) print(rating) muestra cada valor visitado ANTES de comprobar la condición.
# 4) if rating == rating_to_find -> cuando encuentra 4.2 (4ta posición),
    ↳ imprime "Found".
# 5) break corta el bucle inmediatamente, por lo que los valores restantes (3.
    ↳ 8, 3.3, 4.5)
#     nunca se imprimen.
# Resultado esperado:
# 4.3
# 2.5
# 1.7
# 4.2
# Found

```

```

4.3
2.5
1.7
4.2
Found

```

```

[32]: for i in range(10):
        if i % 2 == 0:
            continue # Si el número es par, salta a la siguiente iteración
        print(f'Odd number {i}')

# Explicación línea por línea:
# 1) for i in range(10) recorre 0..9.
# 2) if i % 2 == 0 -> comprueba si i es par (resto de dividir entre 2 es 0).
# 3) continue -> si es par, se salta el resto del cuerpo del bucle (no llega al
    ↳ print) y pasa
#     a la siguiente iteración.
# 4) print(f'Odd number {i}') solo se ejecuta para los números impares.
# Resultado esperado:
# Odd number 1
# Odd number 3
# Odd number 5
# Odd number 7
# Odd number 9

```

```

Odd number 1
Odd number 3
Odd number 5
Odd number 7
Odd number 9

```

```

[33]: for i in range(10):
        if i % 2 != 0:

```

```
print('Numero impar:', end=' ')
print(i)
```

```
# Explicación línea por línea:
# 1) for i in range(10) recorre 0..9.
# 2) if i % 2 != 0 -> True solo para números impares.
# 3) print('Numero impar:', end=' ') imprime el texto sin salto de línea (end='
↳ ' pone un espacio
#     en vez de '\n'), para que el número quede en la misma línea.
# 4) print(i) imprime el número inmediatamente después, en la misma línea.
# Es la versión equivalente al ejemplo anterior pero sin usar 'continue'.
# Resultado esperado:
# Numero impar: 1
# Numero impar: 3
# Numero impar: 5
# Numero impar: 7
# Numero impar: 9
```

```
Numero impar: 1
Numero impar: 3
Numero impar: 5
Numero impar: 7
Numero impar: 9
```

```
[34]: for i in range(5):
      for a in range(2):
          print(f"{i} - {a}")
          if i == 3 and a == 0:
              break

# Explicación línea por línea:
# 1) Bucle externo: i recorre 0,1,2,3,4.
# 2) Bucle interno: a recorre 0,1 para cada valor de i.
# 3) print(f"{i} - {a}") muestra la combinación actual.
# 4) if i == 3 and a == 0: break -> SOLO corta el bucle INTERNO cuando i=3 y a=0
#     (break únicamente afecta al bucle más cercano, no al externo).
#     Por eso cuando i=3, solo se imprime "3 - 0" y no "3 - 1".
# Resultado esperado:
# 0 - 0
# 0 - 1
# 1 - 0
# 1 - 1
# 2 - 0
# 2 - 1
# 3 - 0
# 4 - 0
# 4 - 1
```



```
0 - 0
0 - 1
1 - 0
1 - 1
2 - 0
2 - 1
3 - 0
4 - 0
4 - 1
```

2.8.1 else en bucles

Se ejecuta cuando el bucle termina con normalidad (sin `break`).

```
[35]: lista = [60, 60, 30, 60]
      element_to_find = 60
      for element in lista:
          if element == element_to_find:
              print("found")
              break
      else:
          print("not found")

# Explicación línea por línea:
# 1) lista contiene 60 en varias posiciones; element_to_find = 60.
# 2) for element in lista recorre la lista.
# 3) if element == element_to_find -> se cumple en la primera posición
    ↪ (element=60).
# 4) print("found") y break -> se sale del bucle inmediatamente.
# 5) el 'else' del for NO se ejecuta porque el bucle terminó con un break.
# Resultado esperado: "found"
```

found

```
[36]: lista = [10, 30, 50, 40]
      element_to_find = 30
      found = False
      for element in lista:
          if element == element_to_find:
              found = True
              break
      if found:
          print("dato encontrado")
      else:
          print("not found")

# Explicación línea por línea:
# 1) Se usa una variable "found" (bandera) en lugar del else del for.
```

```
# 2) for element in lista recorre la lista buscando el valor 30.
# 3) Cuando lo encuentra (en la posición 1), pone found=True y corta el bucle
    ↳ con break.
# 4) Fuera del for, un if/else normal decide el mensaje según el valor de
    ↳ 'found'.
# Resultado esperado: "dato encontrado"
```

dato encontrado

2.9 4. List Comprehensions

Permiten construir listas mediante la ejecución repetida de una expresión para cada elemento de un iterable. Sintaxis: [<expresion> for <item> in <iterable>]

```
[37]: resultado = [char*3 for char in "Enrique"]
print(resultado)

# Explicación línea por línea:
# 1) "Enrique" es el iterable; 'char' toma cada carácter uno por uno.
# 2) 'char*3' repite el carácter 3 veces (multiplicación de strings).
# 3) La comprehension construye una lista con esos strings repetidos, en el
    ↳ mismo orden
#     que los caracteres originales.
# Resultado esperado: ['EEE', 'nnn', 'rrr', 'iii', 'qqq', 'uuu', 'eee']
```

```
['EEE', 'nnn', 'rrr', 'iii', 'qqq', 'uuu', 'eee']
```

```
[38]: lista = []
for char in 'Enrique':
    lista.append(char)
print(lista)

# Explicación línea por línea:
# 1) lista vacía.
# 2) for char in 'Enrique' recorre cada carácter del string.
# 3) lista.append(char) agrega cada carácter (sin repetir) a la lista.
# Es el bucle equivalente, sin comprehension, que solo separa el string en
    ↳ caracteres.
# Resultado esperado: ['E', 'n', 'r', 'i', 'q', 'u', 'e']
```

```
['E', 'n', 'r', 'i', 'q', 'u', 'e']
```

```
[39]: resultado = [2 for _ in 'Enrique']
print(resultado)

# Explicación línea por línea:
# 1) 'Enrique' tiene 7 caracteres, así que el for itera 7 veces.
# 2) El guion bajo '_' es una variable "descartable": se usa por convención
    ↳ cuando
```

```
# no nos interesa el valor de la variable de iteración.
# 3) La expresión antes del for (2) no depende del item, así que se repite el
↳ mismo valor.
# Resultado esperado: [2, 2, 2, 2, 2, 2, 2]
```

```
[2, 2, 2, 2, 2, 2, 2]
```

2.9.1 Versión extendida: con filtro if

```
[40]: pares = [y for y in range(9) if y % 2 == 0]
print(pares)

# Explicación línea por línea:
# 1) range(9) genera 0..8.
# 2) 'if y % 2 == 0' filtra solo los valores pares (resto 0 al dividir entre 2).
# 3) Se construye una lista únicamente con esos valores filtrados.
# Resultado esperado: [0, 2, 4, 6, 8]
```

```
[0, 2, 4, 6, 8]
```

```
[41]: pares = []
for x in range(9):
    if x % 2 == 0:
        pares.append(x)
print(pares)

# Explicación línea por línea:
# 1) Bucle equivalente sin comprehension.
# 2) for x in range(9) recorre 0..8.
# 3) if x % 2 == 0 filtra los pares.
# 4) pares.append(x) los agrega a la lista resultado.
# Resultado esperado: [0, 2, 4, 6, 8]
```

```
[0, 2, 4, 6, 8]
```

2.9.2 Versión completa: if/else dentro de la expresión

```
[42]: resultado = [0 if x % 2 == 0 else x for x in range(9)]
print(resultado)

# Explicación línea por línea:
# 1) 'x for x in range(9)' recorre 0..8.
# 2) '0 if x % 2 == 0 else x' es una expresión ternaria: si x es par, el valor
↳ puesto
# en la lista es 0; si es impar, se conserva el valor original de x.
# Resultado esperado: [0, 1, 0, 3, 0, 5, 0, 7, 0]
```

```
[0, 1, 0, 3, 0, 5, 0, 7, 0]
```

```
[43]: resultado = [x**2 if x > 2 else x for x in range(-4, 5) if x > 0]
print(resultado)

# Explicación línea por línea:
# 1) range(-4,5) genera -4,-3,-2,-1,0,1,2,3,4.
# 2) El filtro final 'if x > 0' descarta los valores <= 0, dejando solo 1,2,3,4.
# 3) Para cada valor que pasa el filtro, se aplica la expresión ternaria:
#     si x > 2 -> x**2 (elevado al cuadrado); si no -> x tal cual.
# 4) Para x=1 -> 1 (no >2); x=2 -> 2 (no >2); x=3 -> 9 (3**2); x=4 -> 16 (4**2).
# Resultado esperado: [1, 2, 9, 16]
```

[1, 2, 9, 16]

```
[44]: lista = []
for x in range(-4, 5):
    if x > 0:
        if x > 2:
            lista.append(x**2)
        else:
            lista.append(x)
print(lista)

# Explicación línea por línea:
# 1) Bucle equivalente sin comprehension, con if anidados en vez de ternario.
# 2) Solo procesa los valores x > 0 (filtro).
# 3) Si x > 2, agrega x al cuadrado; en caso contrario agrega x directamente.
# Resultado esperado: [1, 2, 9, 16]
```

[1, 2, 9, 16]

```
[45]: resultado = [x**2 if x > 0 else 100 if x == 0 else x for x in range(-4, 5)]
print(resultado)

# Explicación línea por línea:
# 1) range(-4,5) genera -4,-3,-2,-1,0,1,2,3,4 (sin filtro esta vez).
# 2) La expresión ternaria encadenada dice:
#     - si x > 0 -> x**2
#     - si no, y x == 0 -> 100
#     - si no (x < 0) -> x tal cual (valor negativo original)
# 3) Para x=-4..-1 se conserva el valor negativo; para x=0 -> 100; para x=1..4
    ↪ -> su cuadrado.
# Resultado esperado: [-4, -3, -2, -1, 100, 1, 4, 9, 16]
```

[-4, -3, -2, -1, 100, 1, 4, 9, 16]

```
[46]: lista = []
for x in range(-4, 5):
    if x > 0:
        lista.append(x**2)
```

```

    elif x == 0:
        lista.append(100)
    else:
        lista.append(x)
print(lista)

# Explicación línea por línea:
# 1) Bucle equivalente sin comprehension del ejemplo anterior.
# 2) if x > 0 -> agrega el cuadrado.
# 3) elif x == 0 -> agrega 100.
# 4) else (x < 0) -> agrega el valor original.
# Resultado esperado: [-4, -3, -2, -1, 100, 1, 4, 9, 16]

```

[-4, -3, -2, -1, 100, 1, 4, 9, 16]

2.9.3 Anidamiento de list comprehensions

```

[47]: lista_1 = [1, 2, 3]
      lista_2 = [4, 5]
      resultado = [(x, y) for x in lista_1 for y in lista_2]
      print(resultado)

# Explicación línea por línea:
# 1) lista_1 tiene 3 elementos, lista_2 tiene 2 elementos.
# 2) 'for x in lista_1 for y in lista_2' es equivalente a un bucle anidado: por
    ↳ cada x,
#     se recorren TODOS los valores de y.
# 3) '(x, y)' construye una tupla combinando cada par posible.
# 4) El resultado tiene 3*2 = 6 tuplas, en el orden:
    ↳ (1,4), (1,5), (2,4), (2,5), (3,4), (3,5).
# Resultado esperado: [(1, 4), (1, 5), (2, 4), (2, 5), (3, 4), (3, 5)]

```

[(1, 4), (1, 5), (2, 4), (2, 5), (3, 4), (3, 5)]

2.9.4 Otros tipos de comprehensions: diccionarios y conjuntos

```

[48]: d = {x : x*x for x in range(10)}
      print(d)
      print(type(d))

# Explicación línea por línea:
# 1) range(10) genera 0..9.
# 2) '{x : x*x for x in range(10)}' usa llaves y una expresión "clave:valor" ->
    ↳ crea un diccionario
#     en lugar de una lista.
# 3) Cada clave es x, y su valor es x*x (el cuadrado).
# 4) type(d) confirma que el resultado es un dict.
# Resultado esperado:

```

```
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
# <class 'dict'>
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
<class 'dict'>
```

```
[49]: items = ['Banana', 'Pear', 'Olives']
price = [1.1, 1.4, 2.4]
shopping = {k: v for k, v in zip(items, price)}
print(shopping)

# Explicación línea por línea:
# 1) zip(items, price) empareja cada elemento de 'items' con el elemento
    ↪ correspondiente
# de 'price' en el mismo índice, generando tuplas (nombre, precio).
# 2) La dict comprehension desempaqueta cada tupla en 'k' (clave) y 'v' (valor).
# 3) Se construye un diccionario {nombre_producto: precio}.
# Resultado esperado: {'Banana': 1.1, 'Pear': 1.4, 'Olives': 2.4}
```

```
{'Banana': 1.1, 'Pear': 1.4, 'Olives': 2.4}
```

```
[50]: items = ['Banana', 'Pear', 'Olives']
price = [1.1, 1.4, 2.4]
for k in zip(items, price):
    print(type(k))
    print(k)

# Explicación línea por línea:
# 1) zip(items, price) genera un iterador de tuplas (elemento_items,
    ↪ elemento_price).
# 2) for k in zip(...) recorre cada tupla generada.
# 3) type(k) muestra que cada elemento es de tipo tuple.
# 4) print(k) muestra el contenido de la tupla.
# Resultado esperado:
# <class 'tuple'>
# ('Banana', 1.1)
# <class 'tuple'>
# ('Pear', 1.4)
# <class 'tuple'>
# ('Olives', 2.4)
```

```
<class 'tuple'>
('Banana', 1.1)
<class 'tuple'>
('Pear', 1.4)
<class 'tuple'>
('Olives', 2.4)
```

```
[51]: c = {x for x in range(10)}
print(c)
print(type(c))

# Explicación línea por línea:
# 1) range(10) genera 0..9.
# 2) '{x for x in range(10)}' usa llaves con una expresión simple (no clave:
    ↪ valor) -> crea un SET
#     (conjunto), no un diccionario.
# 3) type(c) confirma que es de tipo set.
# Resultado esperado:
# {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
# <class 'set'>
# Nota: los conjuntos no garantizan un orden específico de impresión, aunque
    ↪ con enteros
# pequeños suele mostrarse en orden ascendente en CPython.
```

```
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
<class 'set'>
```

```
[52]: tupla = tuple(x for x in range(4))
print(tupla)

# Explicación línea por línea:
# 1) 'x for x in range(4)' entre paréntesis de la función tuple() es en
    ↪ realidad un
#     generador (generator expression), no una "tuple comprehension" real
    ↪ (Python no tiene
#     sintaxis nativa de comprehension con paréntesis para tuplas).
# 2) tuple(...) consume ese generador y construye una tupla con sus valores.
# Resultado esperado: (0, 1, 2, 3)
```

```
(0, 1, 2, 3)
```

2.10 5. Excepciones

Alteran el flujo de ejecución convencional. Python las lanza automáticamente ante errores, y el programador puede capturarlas con `try/except` o lanzarlas explícitamente con `raise`.

```
[53]: lista = [6, 1, 0, 5]
lista[4]
print('Código tras el error')

# Explicación línea por línea:
# 1) lista tiene 4 elementos, con índices válidos 0,1,2,3.
# 2) lista[4] intenta acceder a una posición que NO existe (fuera de rango).
# 3) Esto lanza una excepción IndexError, y como no está capturada, el programa
    ↪ se detiene
```

```
# en ese punto; la línea print('Código tras el error') nunca se ejecuta.
# Resultado esperado:
# IndexError: list index out of range
# (el print posterior no llega a ejecutarse)
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[53], line 2
      1 lista = [6, 1, 0, 5]
----> 2 lista[4]
      3 print('Código tras el error')
      4
      5 # Explicación línea por línea:

IndexError: list index out of range
```

```
[54]: lista = [6, 1, 0, 5]
i = 300
try:
    a = lista[i]
except IndexError:
    print('He capturado la excepción de tipo IndexError')
    a = 0
print('Código tras el bloque try')
print(a)

# Explicación línea por línea:
# 1) i = 300, muy fuera del rango válido de la lista (0..3).
# 2) 'try:' intenta ejecutar 'a = lista[i]', que provocará un IndexError.
# 3) 'except IndexError:' captura específicamente ese tipo de error.
# 4) Dentro del except se informa el error y se asigna un valor por defecto
    ↪ a=0, para que
# el programa pueda seguir funcionando sin romperse.
# 5) El código después del try/except se ejecuta normalmente (no se interrumpe
    ↪ el programa).
# Resultado esperado:
# He capturado la excepción de tipo IndexError
# Código tras el bloque try
# 0
```

```
He capturado la excepción de tipo IndexError
Código tras el bloque try
0
```

```
[55]: try:
      4/0
```



```

except:
    print('He capturado la división por cero')

# Explicación línea por línea:
# 1) '4/0' intenta dividir por cero, lo que lanza ZeroDivisionError.
# 2) 'except:' sin especificar tipo captura CUALQUIER excepción (no es lo más
    ↳ recomendable,
#     porque puede ocultar errores inesperados, pero funciona para este ejemplo).
# 3) Se imprime el mensaje del except.
# Resultado esperado: "He capturado la división por cero"

```

He capturado la división por cero

```

[56]: lista = [6, 1, 0, 5]
try:
    print(lista[4])
except Exception as e:
    print("Error = " + str(e))
print('Reacheable Code')

# Explicación línea por línea:
# 1) lista[4] está fuera de rango -> lanza IndexError.
# 2) 'except Exception as e' captura la excepción y la guarda en la variable
    ↳ 'e'.
# 3) str(e) convierte el objeto de excepción en su mensaje de texto (ej: "list
    ↳ index out of range").
# 4) Se concatena con "Error = " y se imprime.
# 5) El programa continúa normalmente después del bloque try/except.
# Resultado esperado:
# Error = list index out of range
# Reacheable Code

```

Error = list index out of range

Reacheable Code

2.10.1 try/finally

```

[57]: lista = [6, 1, 0, 5]
try:
    a = lista[1]
except IndexError:
    print('Exception IndexError Captured')
finally:
    print('Bloque finally')
b = lista[2]
print('Código tras el bloque try')
print(b)

```

```

# Explicación línea por línea:
# 1) lista[1] = 1, un acceso válido, por lo que NO se produce ninguna excepción.
# 2) Como no hay error, el bloque 'except IndexError' no se ejecuta.
# 3) 'finally' SIEMPRE se ejecuta, haya habido excepción o no; por eso se
    ↪ imprime
#     'Bloque finally' de todas formas.
# 4) b = lista[2] = 0 (acceso válido, fuera del try).
# 5) Se imprimen los mensajes finales y el valor de b.
# Resultado esperado:
# Bloque finally
# Código tras el bloque try
# 0

```

Bloque finally
Código tras el bloque try
0

2.10.2 raise

```

[58]: lista = [6, 1, 0, 5]
      indice = 4
      try:
          if indice >= len(lista):
              raise IndexError('Índice fuera de rango')
      except IndexError:
          print('Excepción de tipo IndexError capturada')

# Explicación línea por línea:
# 1) lista tiene longitud 4 (índices válidos 0..3); indice = 4.
# 2) 'if indice >= len(lista)' comprueba manualmente si el índice se saldría de
    ↪ rango
#     ANTES de intentar acceder a la lista.
# 3) 'raise IndexError(...)' lanza explícitamente una excepción de tipo
    ↪ IndexError con
#     un mensaje personalizado, aunque en este caso el acceso real a la lista
    ↪ nunca ocurre.
# 4) El 'except IndexError' captura esa excepción lanzada manualmente.
# Resultado esperado: "Excepción de tipo IndexError capturada"

```

Excepción de tipo IndexError capturada

2.11 5.1 Ejercicios (enunciados)

1. Escribe un programa que calcule la suma de todos los elementos de una lista dada. La lista solo puede contener elementos numéricos.
2. Dada una lista con elementos duplicados, escribir un programa que muestre una nueva lista con el mismo contenido que la primera pero sin elementos duplicados. No se puede usar Set.

3. Escribe un programa que construya un diccionario con un número (entre 1 y n) de elementos de la forma (x, x*x).
4. Dada una lista de palabras, comprobar si alguna empieza por 'a' y tiene más de 9 caracteres. Detener el programa apenas se encuentre.
5. Dada una lista L de números positivos, mostrar la lista de índices i tales que L[i] es múltiplo de 3.
6. Dado un diccionario de pares string-numérico, mostrar la clave con el valor numérico más alto.
7. Dadas las listas a y b, multiplicar cada elemento de a mayor que 5 por cada elemento de b menor que 14.
8. Pedir un valor X al usuario y mostrar el resultado de 10/X, gestionando excepciones.
9. Crear un diccionario y pedir al usuario una clave; mostrar el valor asociado o gestionar el error si no existe.
10. List comprehension: construir una lista con los enteros positivos de una lista dada (descartando floats).
11. Set comprehension: construir un conjunto con las vocales de una palabra dada.
12. List comprehension: números del 0 al 50 que contienen el dígito 3.
13. Dictionary comprehension: tamaños de cada palabra en una frase dada.
14. List comprehension: números del 1 al 10; la primera mitad en formato numérico, la segunda en texto.

2.12 6. Soluciones

```
[59]: numeros = [1, 5, 9, 2, 3]
suma = 0
for numero in numeros:
    suma += numero
print(suma)

# Explicación línea por línea (Ejercicio 1):
# 1) numeros: lista de valores numéricos; suma inicia en 0 (acumulador).
# 2) for numero in numeros recorre cada elemento.
# 3) suma += numero va acumulando el total.
# 4) print(suma) muestra el total: 1+5+9+2+3 = 20.
# Resultado esperado: 20
```

20

```
[60]: numeros = [1, 2, 2, 3, 4, 5, 4, 5, 3, 3, 1]
resultado = []
for numero in numeros:
    if numero not in resultado:
        resultado.append(numero)
print(resultado)

# Explicación línea por línea (Ejercicio 2):
# 1) numeros contiene duplicados.
# 2) resultado empieza vacío.
```

```
# 3) for numero in numeros recorre cada valor.
# 4) 'if numero not in resultado' comprueba si el valor YA fue agregado antes.
# 5) Si no está, se agrega con append; así se evita duplicar elementos, sin
    ↪ usar 'set'.
# Resultado esperado: [1, 2, 3, 4, 5]
```

[1, 2, 3, 4, 5]

```
[61]: n = 5
diccionario = {}
for i in range(1, n+1):
    diccionario[i] = i*i
print(diccionario)

# Explicación línea por línea (Ejercicio 3):
# 1) n = 5; diccionario vacío.
# 2) range(1, n+1) genera 1,2,3,4,5 (incluye n gracias al +1).
# 3) diccionario[i] = i*i asigna a cada clave i su cuadrado como valor.
# Resultado esperado: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

```
[62]: palabras = ["", "Valencia", "python", "asignaturas", "programación", "java",
    ↪ "estudiantes", "académicos"]
for palabra in palabras:
    if len(palabra) >= 9 and palabra[0] == 'a':
        print(f"Palabra encontrada: {palabra}")
        break
    else:
        print("Palabra no encontrada")

# Explicación línea por línea (Ejercicio 4):
# 1) palabras: lista de strings de distinta longitud (incluye un string vacío
    ↪ al inicio).
# 2) for palabra in palabras recorre la lista en orden.
# 3) 'len(palabra) >= 9 and palabra[0] == "a"' comprueba longitud >=9
    ↪ caracteres Y que
#     empiece por la letra 'a' minúscula.
# 4) "asignaturas" (11 letras) es la primera que cumple ambas condiciones -> se
    ↪ imprime y
#     se corta el bucle con break.
# 5) El 'else' del for solo se ejecutaría si NINGUNA palabra cumpliera la
    ↪ condición
#     (no ocurre aquí porque hubo break).
# Resultado esperado: "Palabra encontrada: asignaturas"
```

Palabra encontrada: asignaturas

```
[63]: L = [3, 5, 13, 12, 1, 9]
resultado = []
for i in range(len(L)):
    if L[i] % 3 == 0:
        resultado.append(i)
print(resultado)

# Explicación línea por línea (Ejercicio 5):
# 1) L: lista de números positivos.
# 2) for i in range(len(L)) recorre los índices de la lista (0 a 5).
# 3) 'if L[i] % 3 == 0' comprueba si el valor en esa posición es múltiplo de 3.
# 4) Si lo es, se guarda el ÍNDICE (no el valor) en 'resultado'.
# 5) L[0]=3, L[3]=12, L[5]=9 son múltiplos de 3 -> sus índices 0, 3, 5 se
    ↪ agregan.
# Resultado esperado: [0, 3, 5]
```

[0, 3, 5]

```
[64]: d = {'a': 4.3, 'b': 1, 'c': 7.8, 'd': -5}
maximo = float("-inf") # Inicializamos 'maximo' a -infinito
if len(d) == 0:
    print("El diccionario está vacío")
for clave, valor in d.items():
    if valor > maximo:
        maximo = valor
        resultado = clave
print(resultado)

# Explicación línea por línea (Ejercicio 6):
# 1) d: diccionario con valores numéricos.
# 2) maximo se inicializa en -infinito para que CUALQUIER valor del diccionario
    ↪ sea mayor
#    en la primera comparación.
# 3) Se comprueba si el diccionario está vacío (no es el caso aquí).
# 4) for clave, valor in d.items() recorre cada par clave-valor.
# 5) Si el valor actual supera a 'maximo', se actualizan 'maximo' y 'resultado'
    ↪ (la clave).
# 6) Al final, 'resultado' contiene la clave del valor máximo encontrado (7.8
    ↪ -> clave 'c').
# Resultado esperado: "c"
```

c

```
[66]: a = [2, 4, 6, 8]
b = [7, 11, 15, 22]
for i in a:
    if i > 5:
        for j in b:
```

```

if j < 14:
    print(f"{i} x {j} = {i*j}")

```

Explicación línea por línea (Ejercicio 7):

1) a y b son las listas dadas.

2) for i in a recorre a; el filtro 'if i > 5' deja solo 6 y 8.

3) Para cada uno de esos valores, se recorre b con un for anidado.

4) 'if j < 14' filtra en b solo los valores 7 y 11.

5) Se multiplican e imprimen todas las combinaciones válidas (2 valores de a, 2 de b = 4 líneas).

Resultado esperado:

6 x 7 = 42

6 x 11 = 66

8 x 7 = 56

8 x 11 = 88

6 x 7 = 42

6 x 11 = 66

8 x 7 = 56

8 x 11 = 88

```

[65]: try:
        x = input()
        numero = float(x)
        print(f"El resultado de 10/{x} es: {10/numero}")
    except ValueError:
        print(f"El valor '{x}' no es numérico")
    except ZeroDivisionError:
        print("Error: división por cero")

```

Explicación línea por línea (Ejercicio 8):

1) x = input() lee un valor de texto ingresado por el usuario.

2) numero = float(x) intenta convertirlo a número decimal; si el texto no es numérico

(por ejemplo "abc"), lanza ValueError.

3) Si la conversión es exitosa, se calcula e imprime 10/numero.

4) Si numero es 0, la división lanza ZeroDivisionError, capturado por su except específico.

5) 'except ValueError' captura entradas no numéricas y muestra un mensaje informativo.

Resultado esperado (ejemplo con entrada "2"): "El resultado de 10/2 es: 5.0"

Resultado esperado (ejemplo con entrada "0"): "Error: división por cero"

Resultado esperado (ejemplo con entrada "abc"): "El valor 'abc' no es numérico"

6798

El resultado de 10/6798 es: 0.0014710208884966167

```
[68]: notas = {'Programación': 9.5, "Inteligencia artificial": 7.6, "Redes": 5.2,
↳ "Sistemas operativos": 6.4}
try:
    asignatura = input("Introduce una asignatura")
    print(f"La nota de la asignatura '{asignatura}' es: {notas[asignatura]}")
except KeyError:
    print(f"Asignatura incorrecta: {asignatura}")

# Explicación línea por línea (Ejercicio 9):
# 1) notas: diccionario asignatura -> nota.
# 2) asignatura = input(...) pide al usuario el nombre de una asignatura.
# 3) notas[asignatura] busca esa clave en el diccionario; si no existe, lanza
↳ KeyError.
# 4) Si existe, se imprime la nota correspondiente.
# 5) 'except KeyError' captura el caso de clave inexistente y muestra un
↳ mensaje adecuado.
# Resultado esperado (entrada "Redes"): "La nota de la asignatura 'Redes' es: 5.
↳ 2"
# Resultado esperado (entrada "Química"): "Asignatura incorrecta: Química"
```

Introduce una asignatura Redes

La nota de la asignatura 'Redes' es: 5.2

```
[69]: lista = [1, 4, -3, -1.5, 6.5, 2, 8, 2.1]
resultado = [numero for numero in lista if type(numero) == int and numero > 0]
print(resultado)

# Explicación línea por línea (Ejercicio 10):
# 1) lista mezcla enteros y floats, positivos y negativos.
# 2) La comprehension filtra con 'type(numero) == int' (descarta floats) y
↳ 'numero > 0'
# (descarta negativos y cero).
# 3) Solo 1, 4, 2 y 8 son enteros positivos en la lista.
# Resultado esperado: [1, 4, 2, 8]
```

[1, 4, 2, 8]

```
[70]: palabra = "murcielago"
resultado = {letra for letra in palabra if letra in ('a', 'e', 'i', 'o', 'u')}
print(resultado)

# Explicación línea por línea (Ejercicio 11):
# 1) palabra = "murcielago".
# 2) La comprehension con llaves y expresión simple crea un SET (conjunto), no
↳ una lista.
# 3) 'if letra in (...)' filtra solo las vocales.
```

```
# 4) Al ser un conjunto, no se repiten vocales aunque aparezcan varias veces en
↳ la palabra
# ("murcielago" tiene 'u','i','e','a','o').
# Resultado esperado: {'u', 'i', 'e', 'a', 'o'} (el orden de impresión puede
↳ variar)
```

```
{'o', 'i', 'a', 'e', 'u'}
```

```
[71]: resultado = [numero for numero in range(51) if '3' in str(numero)]
print(resultado)

# Explicación línea por línea (Ejercicio 12):
# 1) range(51) genera 0..50.
# 2) str(numero) convierte cada número a texto para poder buscar el carácter
↳ '3' dentro de él.
# 3) 'if "3" in str(numero)' filtra los números que contienen el dígito 3 en
↳ cualquier posición
# (por ejemplo, 3, 13, 23, pero también 30-39 porque contienen un '3' en las
↳ decenas).
# Resultado esperado: [3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43]
```

```
[3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43]
```

```
[72]: frase = "Soy un ser humano"
resultado = {palabra: len(palabra) for palabra in frase.split()}
print(resultado)

# Explicación línea por línea (Ejercicio 13):
# 1) frase.split() separa la frase en una lista de palabras usando espacios
↳ como delimitador:
# ['Soy', 'un', 'ser', 'humano'].
# 2) La dict comprehension crea un diccionario {palabra:
↳ longitud_de_la_palabra}.
# 3) len(palabra) calcula el número de caracteres de cada palabra.
# Resultado esperado: {'Soy': 3, 'un': 2, 'ser': 3, 'humano': 6}
```

```
{'Soy': 3, 'un': 2, 'ser': 3, 'humano': 6}
```

```
[73]: numero_a_palabra = {6: "seis", 7: "siete", 8: "ocho", 9: "nueve", 10: "diez"}
resultado = [numero if numero <= 5 else numero_a_palabra[numero] for numero in
↳ range(1, 11)]
print(resultado)

# Explicación línea por línea (Ejercicio 14):
# 1) numero_a_palabra: diccionario que traduce números del 6 al 10 a su texto
↳ en español.
# 2) range(1, 11) genera 1..10.
```



```
# 3) La expresión ternaria dice: si numero <= 5, se deja el número tal cual
↳(formato numérico);
# si no (6 a 10), se busca su representación en texto en el diccionario.
# Resultado esperado: [1, 2, 3, 4, 5, 'seis', 'siete', 'ocho', 'nueve', 'diez']
```

```
[1, 2, 3, 4, 5, 'seis', 'siete', 'ocho', 'nueve', 'diez']
```

3 Parte 2: Funciones

3.1 1. ¿Qué son?

- Son un mecanismo que ofrece el lenguaje para agrupar conjuntos de instrucciones de manera que se puedan ejecutar cuando el programador considere oportuno.
- Las funciones van identificadas por un nombre.
- Se debe usar este nombre para invocar (ejecutar) el código de la función.
- Las funciones calculan un resultado (output) en base a unos parámetros de entrada (input).
- En cada invocación de la función, los parámetros de entrada pueden variar.

```
[74]: def sumar(x, y):
        suma = x + y
        return suma

print(sumar(4, 5))
print(sumar(-2, 0))

# Explicación línea por línea:
# 1) def sumar(x, y): define una función que recibe dos parámetros.
# 2) suma = x + y calcula la suma dentro del cuerpo de la función.
# 3) return suma devuelve ese resultado a quien llamó a la función.
# 4) print(sumar(4,5)) invoca la función con 4 y 5 -> 4+5=9.
# 5) print(sumar(-2,0)) invoca con -2 y 0 -> -2+0=-2.
# Resultado esperado:
# 9
# -2
```

```
9
```

```
-2
```

```
[75]: c = 30
print(c * 1.793 + 32) # celsius to fahrenheit
c = 26
print(c * 1.793 + 32)

# Explicación línea por línea:
# 1) c = 30 (grados Celsius).
# 2) 'c * 1.793 + 32' aplica la fórmula (aproximada) de conversión a Fahrenheit
↳y se imprime.
# 3) c se reasigna a 26 y se repite el cálculo.
```

```
# Esta celda motiva por qué conviene encapsular la fórmula en una función (para
↪no repetir código).
# Resultado esperado:
# 85.78999999999999
# 78.618
```

```
85.78999999999999
78.618
```

```
[80]: def convert_celsius_to_fahrenheit(c):
        return c * 1.791117 + 32

# Explicación línea por línea:
# 1) Se define una función que recibe 'c' (grados Celsius).
# 2) Devuelve directamente el resultado de la conversión con la fórmula
↪ajustada.
# Esta celda solo define la función; no produce salida por sí sola.
# Resultado esperado: (ninguna salida, solo queda definida la función)
```

```
[81]: degrees = 15
print(convert_celsius_to_fahrenheit(degrees))

# Explicación línea por línea:
# 1) degrees = 15 (grados Celsius a convertir).
# 2) Se invoca la función definida antes, pasando 15 como argumento.
# 3) 15 * 1.791117 + 32 = 58.866755.
# Resultado esperado: 58.866755
```

```
58.866755
```

3.2 2. Ventajas de las funciones

- Eliminación de duplicación de código.
- Incremento de la reutilización de código.
- Manejo de complejidad: descomponer sistemas grandes en piezas más pequeñas y comprensibles.

3.3 3. Programación de funciones en Python

```
def name(arg1, arg2, ..., argN):
    statements
```

- Las funciones son instrucciones convencionales que se ejecutan cuando el flujo de control llega a ellas.
- La definición de la función debe ejecutarse antes de poder invocarla.

```
[84]: def crear_diccionario(keys, values):
        return dict(zip(keys, values))
```

```
# Explicación línea por línea:
# 1) Se define una función que recibe dos listas: 'keys' y 'values'.
# 2) zip(keys, values) empareja cada clave con su valor correspondiente por
    ↪ posición.
# 3) dict(...) convierte esos pares en un diccionario.
# 4) return devuelve ese diccionario al invocador.
# Resultado esperado: (ninguna salida, solo queda definida la función)
```

```
[85]: nombres = ['Alfred', 'James', 'Peter', 'Harvey']
      edades = [87, 43, 19, 16]
      usuarios = crear_diccionario(nombres, edades)
      print(usuarios)

# Explicación línea por línea:
# 1) nombres y edades son dos listas paralelas (mismo orden/misma longitud).
# 2) Se invoca crear_diccionario, definida en la celda anterior, pasando ambas
    ↪ listas.
# 3) Internamente se construye {nombre: edad} para cada par.
# 4) print(usuarios) muestra el diccionario resultante.
# Resultado esperado: {'Alfred': 87, 'James': 43, 'Peter': 19, 'Harvey': 16}
```

```
{'Alfred': 87, 'James': 43, 'Peter': 19, 'Harvey': 16}
```

- Las funciones pueden estar anidadas en sentencias condicionales o bucles.

```
[86]: a = -2
      if a > 0:
          def operacion(x, y):
              return x + y
      else:
          def operacion(x, y):
              return x * y

      print(operacion(3, 4))

# Explicación línea por línea:
# 1) a = -2.
# 2) if a > 0 -> False, por lo que se define 'operacion' dentro del bloque
    ↪ 'else'.
# 3) La versión de 'operacion' definida en el else multiplica sus argumentos (x
    ↪ * y).
# 4) print(operacion(3,4)) invoca esa versión: 3*4 = 12.
# Resultado esperado: 12
```

12

- Argumentos y return son opcionales.

```
[87]: def print_hola_mundo():
        print('Hola Mundo')
        return # es opcional pero mejora la legibilidad

print_hola_mundo()

# Explicación línea por línea:
# 1) Se define una función sin parámetros.
# 2) print('Hola Mundo') imprime el saludo.
# 3) 'return' sin valor simplemente termina la función (equivale a devolver
    ↪ None); se usa
#     aquí solo por claridad, no es obligatorio.
# 4) Se invoca la función.
# Resultado esperado: "Hola Mundo"
```

Hola Mundo

```
[88]: def print_hola_mundo():
        print('Hola Mundo')
        # return None

valor = print_hola_mundo()
print(valor)

# Explicación línea por línea:
# 1) La función solo imprime el texto, sin sentencia return explícita.
# 2) Cuando una función no tiene 'return', Python devuelve automáticamente None.
# 3) valor = print_hola_mundo() ejecuta la función (imprime "Hola Mundo") y
    ↪ guarda su
#     valor de retorno (None) en 'valor'.
# 4) print(valor) muestra ese None.
# Resultado esperado:
# Hola Mundo
# None
```

Hola Mundo

None

- La sentencia `return` puede aparecer en cualquier parte de la función.

```
[89]: def div_safe(x, y):
        if y == 0:
            return "hola"
        return x / y

print(div_safe(6, 2))
print(div_safe(6, 0))
a = div_safe(6, 0)
```

```

print(type(a))

# Explicación línea por línea:
# 1) div_safe(x, y): si y es 0, retorna el string "hola" de inmediato (return
    ↪temprano).
# 2) Si y no es 0, se llega a la segunda línea 'return x / y', que hace la
    ↪división normal.
# 3) div_safe(6,2) -> y=2 (no es 0) -> retorna 6/2 = 3.0.
# 4) div_safe(6,0) -> y=0 -> retorna "hola" (nunca llega a dividir).
# 5) a = div_safe(6,0) guarda "hola"; type(a) confirma que es un str.
# Resultado esperado:
# 3.0
# hola
# <class 'str'>

```

```

3.0
hola
<class 'str'>

```

```

[90]: def div_safe(x, y):
        if y != 0:
            return x / y
        else:
            return 0

print(div_safe(6, 0))

# Explicación línea por línea:
# 1) Versión alternativa: si y es distinto de 0, retorna la división normal.
# 2) Si y es 0 (else), retorna 0 en lugar de un string, evitando el error de
    ↪división por cero.
# 3) div_safe(6,0) -> y=0 -> entra al else -> retorna 0.
# Resultado esperado: 0

```

```
0
```

- Puede haber más de un valor de retorno. Se devuelve una tupla.

```

[91]: def devolver_primeros_tres(coleccion):
        return coleccion[0], coleccion[1], coleccion[2]

t = devolver_primeros_tres(['Alfred', 'James', 'Peter', 'Harvey'])
print(t)
print(t[1])
print(type(t))

# Explicación línea por línea:

```

```
# 1) La función retorna tres valores separados por comas -> Python los
    ↪empaqueta en una tupla.
# 2) Se invoca con una lista de 4 elementos; se devuelven los 3 primeros
    ↪('Alfred', 'James', 'Peter').
# 3) print(t) muestra la tupla completa.
# 4) t[1] accede al segundo elemento de la tupla ('James').
# 5) type(t) confirma que el resultado es una tupla.
# Resultado esperado:
# ('Alfred', 'James', 'Peter')
# James
# <class 'tuple'>
```

```
('Alfred', 'James', 'Peter')
James
<class 'tuple'>
```

```
[92]: primero, segundo, tercero = devolver_primer_segundo(['Alfred', 'James',
    ↪'Peter', 'Harvey'])
print(primer)
print(segundo)
print(tercero)

# Explicación línea por línea:
# 1) Se invoca la misma función, que retorna una tupla de 3 elementos.
# 2) 'primero, segundo, tercero = ...' desempaqueta automáticamente la tupla en
    ↪tres variables
#     individuales, en el mismo orden en que fueron retornadas.
# 3) Se imprime cada variable por separado.
# Resultado esperado:
# Alfred
# James
# Peter
```

```
Alfred
James
Peter
```

- El tipo de los parámetros y el valor de retorno se pueden especificar (annotations), pero son simples “hints” (sugerencias), no se validan en tiempo de ejecución.

```
[93]: def add2(x: int, y: int) -> int:
        return x + y

print(add2(1, 2))
print(add2("aa", "ee"))

# Explicación línea por línea:
```

```
# 1) 'x: int, y: int' y '-> int' son anotaciones de tipo: sugieren que se
↳esperan enteros
# y que se devolverá un entero, pero Python NO fuerza esta restricción en
↳tiempo de ejecución.
# 2) add2(1,2) -> 1+2 = 3 (enteros reales, como sugieren las anotaciones).
# 3) add2("aa","ee") -> como no hay validación real, el operador '+' entre
↳strings concatena:
# "aa"+"ee" = "aeee". Python lo permite igualmente, aunque contradiga la
↳anotación.
# Resultado esperado:
# 3
# aeee
```

3

aeee

- Los argumentos pueden tener valores por defecto, siempre al final.

```
[94]: def f(a, b, c=None):
        if a is not None:
            print('Se ha proporcionado a')
        if b is not None:
            print('Se ha proporcionado b')
        if c is not None:
            print('Se ha proporcionado c')

f(1, 5, 2)

# Explicación línea por línea:
# 1) 'c=None' es un valor por defecto: si no se pasa 'c' al llamar, tomará None
↳automáticamente.
# 2) f(1, 5, 2) pasa explícitamente los tres argumentos: a=1, b=5, c=2.
# 3) Cada 'if ... is not None' comprueba si el argumento fue proporcionado (no
↳es None).
# 4) Como los tres valores son distintos de None, se imprimen los tres mensajes.
# Resultado esperado:
# Se ha proporcionado a
# Se ha proporcionado b
# Se ha proporcionado c
```

Se ha proporcionado a

Se ha proporcionado b

Se ha proporcionado c

- Los parámetros se pasan por posición, pero el orden se puede alterar especificando el nombre del parámetro en la llamada (named/key word arguments).

```
[95]: def power(base, exponente):
        return pow(base, exponente)

print(power(2, 3))
print(power(exponente=3, base=2))

# Explicación línea por línea:
# 1) power(base, exponente) calcula base elevado a exponente usando la función
    ↪ pow().
# 2) power(2, 3) pasa los argumentos por posición: base=2, exponente=3 -> 2**3
    ↪ = 8.
# 3) power(exponente=3, base=2) los pasa por NOMBRE, en orden distinto; Python
    ↪ los asigna
#     correctamente según el nombre, no la posición -> mismo resultado.
# Resultado esperado:
# 8
# 8
```

8

8

- Al ejecutarse una sentencia `def`, se crea un objeto de tipo función asociado a un nombre. Las variables pueden referenciar objetos de tipo función.

```
[96]: def sumar(x, y):
        return x + y

f = sumar
print(id(f))
print(id(sumar))
print(type(f))
print(sumar(5, 6))
print(f(5, 6))

# Explicación línea por línea:
# 1) sumar es una función normal.
# 2) f = sumar NO ejecuta la función; simplemente hace que 'f' referencie al
    ↪ MISMO objeto
#     función que 'sumar' (las funciones son objetos como cualquier otro en
    ↪ Python).
# 3) id(f) e id(sumar) deberían ser iguales, porque apuntan al mismo objeto en
    ↪ memoria.
# 4) type(f) confirma que es de tipo 'function'.
# 5) sumar(5,6) y f(5,6) ambas ejecutan el mismo código -> 5+6=11 en ambos
    ↪ casos.
# Resultado esperado (los id concretos varían en cada ejecución, pero coinciden
    ↪ entre sí):
```



```
# <mismo número para id(f) e id(sumar)>
# <mismo número, igual al anterior>
# <class 'function'>
# 11
# 11
```

```
2725123527584
2725123527584
<class 'function'>
11
11
```

```
[97]: def clean_strings(strings, ops):
        result = []
        for value in strings:
            for function in ops:
                value = function(value)
            result.append(value)
        return result

def concat5(x):
    return x + '_5'

strings = [' valencia ', ' barcelona', ' bilbao ']
print(strings)
operations = [str.strip, str.title, concat5]
print(clean_strings(strings, operations))

# Explicación línea por línea:
# 1) clean_strings recibe una lista de strings y una lista de funciones ('ops').
# 2) Para cada string, aplica TODAS las funciones de 'ops' en orden,
    ↳ encadenando el resultado
#     (value = function(value) va transformando el mismo valor paso a paso).
# 3) concat5 agrega el sufijo '_5' a un string.
# 4) operations = [str.strip, str.title, concat5]: primero quita espacios
    ↳ (strip), luego
#     capitaliza cada palabra (title), y finalmente agrega '_5'.
# 5) Para ' valencia ' -> strip -> 'valencia' -> title -> 'Valencia' -> concat5
    ↳ -> 'Valencia_5'.
#     Lo mismo aplica a los otros dos strings.
# Resultado esperado:
# [' valencia ', ' barcelona', ' bilbao ']
# ['Valencia_5', 'Barcelona_5', 'Bilbao_5']
```

```
[' valencia ', ' barcelona', ' bilbao ']
['Valencia_5', 'Barcelona_5', 'Bilbao_5']
```

- Se pueden crear placeholders con la palabra clave `pass`, que representa una sentencia que no

hace nada.

```
[98]: def mi_funcion():
      pass # TODO implement this

mi_funcion()

# Explicación línea por línea:
# 1) 'pass' es un marcador de posición: permite que la función esté
    ↪ sintácticamente
#    completa aunque su cuerpo esté vacío (útil cuando aún no se ha
    ↪ implementado la lógica).
# 2) Se invoca la función, que no hace absolutamente nada.
# Resultado esperado: (ninguna salida)
```

3.4 4. Paso de parámetros

- Tipos simples (inmutables) por valor: `int`, `float`, `string`. Los cambios dentro de la función NO afectan fuera.

```
[99]: def cambiar_valor(x):
      x += 3
      print('Dentro de la función:', x)
      return

a = 2
cambiar_valor(a)
print('Fuera de la función:', a)

# Explicación línea por línea:
# 1) a = 2 (entero, tipo inmutable).
# 2) Al llamar cambiar_valor(a), dentro de la función 'x' referencia
    ↪ inicialmente al mismo
#    objeto que 'a' (el 2).
# 3) 'x += 3' NO modifica el 2 original (los enteros son inmutables); en
    ↪ realidad crea un
#    NUEVO objeto (5) y hace que 'x' pase a referenciar ese nuevo objeto. 'a'
    ↪ sigue apuntando al 2.
# 4) print dentro de la función muestra 5 (el nuevo valor de x).
# 5) Fuera de la función, 'a' no cambió: sigue siendo 2.
# Resultado esperado:
# Dentro de la función: 5
# Fuera de la función: 2
```

Dentro de la función: 5

Fuera de la función: 2

- Tipos complejos (mutables) por referencia: `list`, `set`, `dictionary`.

```
[100]: def anyadir_2_a(x):
        x.append(4)

lista = [0, 1, 3]
anyadir_2_a(lista)
print(lista)

# Explicación línea por línea:
# 1) lista = [0,1,3] (lista, tipo mutable).
# 2) Al llamar anyadir_2_a(lista), 'x' dentro de la función referencia al MISMO
    ↪ objeto lista
#     (no una copia).
# 3) x.append(4) modifica ese objeto compartido directamente.
# 4) Como es el mismo objeto, el cambio SÍ se ve reflejado fuera de la función.
# Resultado esperado: [0, 1, 3, 4]
```

[0, 1, 3, 4]

```
[101]: def anyadir_2_a(x):
        print(id(x))
        return

lista = (0, 1)
print(id(lista))
anyadir_2_a(lista)
print(lista)

# Explicación línea por línea:
# 1) lista es una tupla (0,1) -- tipo inmutable.
# 2) print(id(lista)) muestra la dirección de memoria del objeto tupla.
# 3) Al pasarla a la función, 'x' referencia AL MISMO objeto (mismo id) porque
    ↪ Python pasa
#     siempre la referencia al objeto, sin importar si es mutable o no; lo que
    ↪ cambia es que
#     los tipos inmutables no se pueden modificar "in place".
# 4) print(lista) confirma que la tupla original no cambió (no se modificó
    ↪ nada, la función
#     solo imprimió el id).
# Resultado esperado (los valores de id concretos varían, pero deben coincidir
    ↪ entre sí):
# <id de la tupla>
# <mismo id, mostrado dentro de la función>
# (0, 1)
```

2725122390464

2725122390464

(0, 1)

- Si se quiere “modificar” un objeto de tipo básico (pasado por valor), se puede devolver el resultado y reasignar.

```
[102]: def multiplicar_por_2(x):
        x = x * 2
        return x

a = 3
a = multiplicar_por_2(a)
print(a)

# Explicación línea por línea:
# 1) a = 3 (entero).
# 2) Al invocar multiplicar_por_2(a), dentro de la función x=3*2=6 (nuevo
    ↪ objeto local, no
    ↪ afecta a 'a' fuera de la función).
# 3) return x devuelve ese 6.
# 4) a = multiplicar_por_2(a) REASIGNA 'a' con el valor devuelto (6); así es
    ↪ como se logra
#     "modificar" un valor inmutable desde fuera.
# Resultado esperado: 6
```

6

- Si se quiere que NO se modifique un objeto complejo (pasado por referencia), se puede pasar una copia a la función.

```
[103]: def anyadir_2_a(x):
        x.append(2)
        print('Dentro de la función: ', x)

lista = [0, 1]
anyadir_2_a(lista.copy())
print('Fuera de la función: ', lista)

# Explicación línea por línea:
# 1) lista = [0, 1].
# 2) lista.copy() crea una COPIA superficial (shallow copy) independiente de la
    ↪ lista original.
# 3) Esa copia es la que se pasa a la función; x.append(2) modifica solo la
    ↪ copia.
# 4) Como la función recibió una copia (objeto distinto), la lista original
    ↪ 'lista' NO cambia.
# Resultado esperado:
# Dentro de la función:  [0, 1, 2]
# Fuera de la función:  [0, 1]
```

Dentro de la función: [0, 1, 2]

Fuera de la función: [0, 1]

```
[104]: def anyadir_2_a(x):
        x.append(2)
        print('Dentro de la función: ', x)

lista = [0, 1]
anyadir_2_a(lista[:])
print('Fuera de la función: ', lista)

# Explicación línea por línea:
# 1) 'lista[:]' es slicing completo, que crea otra forma de copia superficial
#    ↪ de la lista
#    (equivalente en efecto a lista.copy()).
# 2) Se pasa esa copia a la función, que le agrega el elemento 2 SOLO a la
#    ↪ copia.
# 3) La lista original queda intacta fuera de la función.
# Resultado esperado:
# Dentro de la función:  [0, 1, 2]
# Fuera de la función:  [0, 1]
```

Dentro de la función: [0, 1, 2]

Fuera de la función: [0, 1]

- Si la lista tiene sublistas anidadas, hay que hacer un deepcopy.

```
[105]: import copy

lista = [2, 4, 16, 32, [34, 10, [5, 5]]]
copia = copy.deepcopy(lista)
copia[0] = 454
copia[4][2][0] = 64
print(lista)
print(copia)
print(f"{id(lista)} - {id(copia)}")
print(f"{id(lista[4])} - {id(copia[4])}")

# Explicación línea por línea:
# 1) lista contiene una sublista anidada (con otra sublista dentro de ella).
# 2) copy.deepcopy(lista) crea una copia PROFUNDA: no solo copia la lista
#    ↪ externa, sino
#    también TODAS las sublistas internas, de forma recursiva, como objetos
#    ↪ totalmente
#    independientes (a diferencia de .copy(), que solo copiaría la referencia a
#    ↪ las sublistas).
# 3) copia[0] = 454 modifica solo la copia en el primer nivel.
# 4) copia[4][2][0] = 64 modifica un valor dentro de la sublista más profunda
#    ↪ de la COPIA.
# 5) Como el deepcopy generó objetos totalmente nuevos en cada nivel, la
#    ↪ 'lista' original
```

```
# permanece completamente intacta, incluidas sus sublistas anidadas.
# 6) Los id de 'lista' y 'copia' son distintos, y también lo son los id de sus
↳ sublistas
# internas (lista[4] vs copia[4]), confirmando que son objetos
↳ independientes.
# Resultado esperado:
# [2, 4, 16, 32, [34, 10, [5, 5]]]
# [454, 4, 16, 32, [34, 10, [64, 5]]]
# <id_lista> - <id_copia> (números distintos)
# <id_lista[4]> - <id_copia[4]> (números distintos)
```

```
[2, 4, 16, 32, [34, 10, [5, 5]]]
[454, 4, 16, 32, [34, 10, [64, 5]]]
2725123565184 - 2725123564800
2725123561216 - 2725123565120
```

- Reasignar un parámetro nunca afecta al objeto de fuera.

```
[106]: def funcion_poco_util(x):
        print(id(x))
        x = [2, 3]
        x.append(4)
        print(id(x))

lista = [0, 1]
print(id(lista))
funcion_poco_util(lista)
print(lista)

# Explicación línea por línea:
# 1) lista = [0,1]; se imprime su id.
# 2) Dentro de la función, el primer print(id(x)) muestra el MISMO id que
↳ 'lista' (x referencia
# al mismo objeto al entrar a la función).
# 3) 'x = [2, 3]' REASIGNA x a un objeto COMPLETAMENTE NUEVO; esto rompe la
↳ conexión con
# la lista original. A partir de aquí, x ya no apunta a 'lista'.
# 4) x.append(4) modifica ese nuevo objeto [2,3,4], sin afectar a 'lista'.
# 5) El segundo print(id(x)) muestra un id DISTINTO al de 'lista' (por la
↳ reasignación).
# 6) Al salir de la función, 'lista' permanece sin cambios: [0, 1].
# Resultado esperado:
# <id_lista>
# <id_lista> (igual al anterior, antes de la reasignación)
# <id_nuevo, distinto>
# [0, 1]
```

```
2725122505984
```

```
2725122505984
2725123561216
[0, 1]
```

```
[107]: def modif_tupla(a):
        print(id(a))
        t = (1, 2)
        print(id(t))

t = (1, 2)
print(id(t))
modif_tupla(t)
print(t)

# Explicación línea por línea:
# 1) t = (1, 2) (tupla externa); se imprime su id.
# 2) Al invocar modif_tupla(t), dentro de la función 'a' referencia
    ↪ inicialmente ese mismo
#     objeto (mismo id se imprime primero).
# 3) 't = (1, 2)' DENTRO de la función crea una variable LOCAL nueva llamada
    ↪ 't' (con el
#     mismo contenido, pero puede o no ser el mismo objeto en memoria según el
    ↪ "interning"
#     de Python para tuplas pequeñas); esta 't' local es independiente de la 't'
    ↪ externa.
# 4) La tupla externa 't' nunca se modifica (las tuplas son inmutables, y aquí
    ↪ solo se creó
#     una variable local con el mismo nombre, sin relación de escritura con la
    ↪ externa).
# Resultado esperado:
# <id_t_externa>
# <mismo id, dentro de la función, antes de la reasignación local>
# <id de la t local, puede coincidir o no con la externa por optimizaciones
    ↪ internas>
# (1, 2)
```

```
2725122389568
2725122389568
2725122422080
(1, 2)
```

3.5 5. Argumentos arbitrarios

- No se sabe a priori cuántos elementos se reciben; se reciben como una tupla.
- Se especifican con el símbolo *.

```
[108]: def saludar(*names):
        print(type(names))
```

```

    for name in names:
        print("Hola " + name)

saludar()
saludar("Manolo", "Pepe", "Luis", "Alex", "Juan")

# Explicación línea por línea:
# 1) '*names' recoge CUALQUIER cantidad de argumentos posicionales en una tupla
    ↳ llamada 'names'.
# 2) type(names) confirma que siempre es de tipo tuple, sin importar cuántos
    ↳ argumentos se pasen.
# 3) for name in names recorre cada nombre recibido e imprime un saludo.
# 4) saludar() se llama sin argumentos -> names es una tupla vacía () -> el for
    ↳ no imprime nada.
# 5) saludar("Manolo", ...) se llama con 5 argumentos -> names =
    ↳ ('Manolo', 'Pepe', 'Luis', 'Alex', 'Juan').
# Resultado esperado:
# <class 'tuple'>
# <class 'tuple'>
# Hola Manolo
# Hola Pepe
# Hola Luis
# Hola Alex
# Hola Juan

```

```

<class 'tuple'>
<class 'tuple'>
Hola Manolo
Hola Pepe
Hola Luis
Hola Alex
Hola Juan

```

```

[109]: def consulta_sql(**kwargs):
        print(kwargs)
        sql = "SELECT * FROM bicis WHERE "
        args = [f"{key}={value}" for key, value in kwargs.items()]
        sql += " AND ".join(args)
        return sql

print(consulta_sql(station='alameda', free=10))

# Explicación línea por línea:
# 1) '**kwargs' recoge cualquier cantidad de argumentos con NOMBRE (keyword
    ↳ arguments) en
#     un diccionario llamado 'kwargs'.
# 2) print(kwargs) muestra ese diccionario tal cual se recibió.

```



```

# 3) La comprehension arma una lista de strings "clave=valor" por cada par en
↳kwargs.
# 4) " AND ".join(args) une esos strings con " AND " entre cada uno.
# 5) Se concatena al SQL base y se retorna la consulta completa.
# 6) consulta_sql(station='alameda', free=10) -> kwargs = {'station':
↳'alameda', 'free':10}.
# Resultado esperado:
# {'station': 'alameda', 'free': 10}
# SELECT * FROM bicis WHERE station=alameda AND free=10

```

```
{'station': 'alameda', 'free': 10}
```

```
SELECT * FROM bicis WHERE station=alameda AND free=10
```

3.5.1 5.1 Desempaquetado en funciones

```

[110]: def saludar(quien, mensaje, gesto):
        print(f"Hola {quien}, {mensaje}, mira esto: {gesto}")

ls_saludo = ['Manolo', 'te quiere saludar', 'args']
saludar(ls_saludo[0], ls_saludo[1], ls_saludo[2])
saludar(*ls_saludo)

# Explicación línea por línea:
# 1) saludar recibe 3 parámetros posicionales normales.
# 2) La primera llamada pasa cada elemento de la lista manualmente por índice.
# 3) La segunda llamada usa '*ls_saludo', que DESEMPAQUETA automáticamente los
↳3 elementos
# de la lista como argumentos posicionales separados (equivalente a la línea
↳anterior,
# pero más compacto).
# 4) Ambas llamadas producen exactamente el mismo resultado.
# Resultado esperado:
# Hola Manolo, te quiere saludar, mira esto: args
# Hola Manolo, te quiere saludar, mira esto: args

```

```
Hola Manolo, te quiere saludar, mira esto: args
```

```
Hola Manolo, te quiere saludar, mira esto: args
```

```

[111]: def saludar(quien, mensaje, gesto):
        print(f"Hola {quien}, {mensaje}, mira esto: {gesto}")

dc_saludo = {
    'quien': [12, 12, 3, 4, 5, 3],
    'gesto': 'guiño',
    'mensaje': 'te quiere enseñar esto'
}
saludar(**dc_saludo)

```

```

# Explicación línea por línea:
# 1) dc_saludo es un diccionario cuyas claves coinciden EXACTAMENTE con los
↳ nombres de los
#   parámetros de la función ('quien', 'mensaje', 'gesto').
# 2) '**dc_saludo' desempaqueta el diccionario como argumentos de tipo keyword
↳ (nombre=valor),
#   sin importar el orden en que estén definidos en el diccionario.
# 3) 'quien' toma el valor de la lista [12,12,3,4,5,3] (se inserta tal cual en
↳ el f-string).
# Resultado esperado:
# Hola [12, 12, 3, 4, 5, 3], te quiere enseñar esto, mira esto: guiño

```

Hola [12, 12, 3, 4, 5, 3], te quiere enseñar esto, mira esto: guiño

3.6 6. Recursividad

Funciones que se llaman a sí mismas.

```

[112]: def factorial(n):
        if n == 0:
            return 1
        else:
            return n * factorial(n-1)

print(factorial(10))

# Explicación línea por línea:
# 1) Caso base: si n==0, la función retorna 1 (el factorial de 0 es 1),
↳ deteniendo la recursión.
# 2) Caso recursivo: si n>0, retorna n multiplicado por el factorial de (n-1);
↳ esto genera
#   llamadas anidadas: factorial(10) = 10 * factorial(9) = 10 * 9 *
↳ factorial(8) ... hasta
#   llegar al caso base.
# 3) Al final se van "deshaciendo" las multiplicaciones: 10*9*8*7*6*5*4*3*2*1*1
↳ = 3628800.
# Resultado esperado: 3628800

```

3628800

```

[113]: def fact(n):
        a = 1
        for i in range(1, n+1):
            a *= i
        return a

print(fact(10))

```

```
# Explicación línea por línea:
# 1) Versión ITERATIVA (sin recursión) del factorial.
# 2) a = 1 (acumulador, valor neutro de la multiplicación).
# 3) for i in range(1, n+1) recorre 1,2,...,n.
# 4) a *= i va multiplicando el acumulador por cada número.
# 5) Al terminar el bucle, a contiene 1*2*3*...*10 = 3628800.
# Resultado esperado: 3628800
```

3628800

```
[114]: fact = lambda x: 1 if x == 0 else x * fact(x-1)
fact(10)

# Explicación línea por línea:
# 1) Se define una función lambda recursiva, asignada a la variable 'fact'.
# 2) 'lambda x: 1 if x == 0 else x * fact(x-1)' es la versión lambda del
    ↪ factorial:
#     caso base x==0 -> 1; caso recursivo -> x * fact(x-1) (la lambda se
    ↪ referencia a sí
#     misma a través del nombre 'fact', que ya existe como variable global al
    ↪ momento de llamarla).
# 3) fact(10) calcula 10! igual que las versiones anteriores.
# Resultado esperado (como última expresión de la celda, en un notebook se
    ↪ muestra el valor): 3628800
```

[114]: 3628800

3.7 7. Funciones Lambda

- Es posible definir funciones anónimas (sin nombre).
- Son expresiones, por lo que pueden aparecer donde una sentencia `def` no puede (por ejemplo, dentro de una lista o como parámetro de otra función).

Formato general: `lambda <argumentos> : <valor a retornar>`

```
[115]: def suma(x, y, z):
        return x + y + z

print(suma(1, 2, 3))

# Ejemplo: función suma como expresión lambda.
suma2 = lambda x, y, z: x + y + z
print(suma2(1, 2, 3))

# Explicación línea por línea:
# 1) 'suma' es una función convencional definida con def, que suma 3 argumentos.
# 2) print(suma(1,2,3)) -> 1+2+3=6.
```

```
# 3) 'suma2' es una lambda equivalente: mismos parámetros, misma expresión de
↳ retorno,
# pero sin la palabra 'return' (es implícita en las lambdas) y sin nombre
↳ propio de función
# (aunque aquí se asigna a una variable para poder invocarla por nombre).
# 4) print(suma2(1,2,3)) da el mismo resultado que la versión con def.
# Resultado esperado:
# 6
# 6
```

6

6

```
[116]: cities = ['Valencia', 'Lugo', 'Barcelona', 'Madrid']
cities.sort()
print(cities)
cities.sort(key=lambda x: len(x))
print(cities)
cities.sort(key=len)
print(cities)

def longitud(x):
    return len(x)

cities.sort(key=longitud)
print(cities)

# Explicación línea por línea:
# 1) cities.sort() ordena la lista ALFABÉTICAMENTE (orden por defecto de
↳ strings).
# 2) cities.sort(key=lambda x: len(x)) ordena según la LONGITUD de cada string
↳ (de más corta
# a más larga), usando una lambda como función 'key' (se aplica a cada
↳ elemento antes de comparar).
# 3) cities.sort(key=len) logra exactamente lo mismo, pasando directamente la
↳ función built-in
# 'len' como key (sin necesidad de lambda, porque len ya hace justo lo que
↳ la lambda hacía).
# 4) 'longitud' es una función convencional equivalente a la lambda anterior;
↳ sort(key=longitud)
# produce el mismo resultado.
# 5) Nota: cuando varios elementos tienen la misma longitud, Python mantiene su
↳ orden relativo
# original (sort estable), así que el orden entre 'Lugo' y 'Madrid' (ambos
↳ con 4-5 letras)
# dependerá de sus posiciones anteriores.
# Resultado esperado:
```

```
# ['Barcelona', 'Lugo', 'Madrid', 'Valencia']
# ['Lugo', 'Madrid', 'Valencia', 'Barcelona']
# ['Lugo', 'Madrid', 'Valencia', 'Barcelona']
# ['Lugo', 'Madrid', 'Valencia', 'Barcelona']
```

```
['Barcelona', 'Lugo', 'Madrid', 'Valencia']
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
```

```
[117]: cities = ['Valencia', 'Lugo', 'Barcelona', 'Madrid']
```

```
def num_caracteres(x):
    return len(x)
```

```
cities.sort(key=num_caracteres)
print(cities)
```

```
# Explicación línea por línea:
# 1) Mismo ejemplo que el anterior pero SIN usar lambda en ningún momento.
# 2) num_caracteres(x) es una función convencional que retorna la longitud del
    ↪ string.
# 3) cities.sort(key=num_caracteres) ordena la lista según esa longitud.
# Resultado esperado: ['Lugo', 'Madrid', 'Valencia', 'Barcelona']
```

```
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
```

- Se puede guardar una referencia a una lambda para utilización repetida.

```
[118]: cities = ['Valencia', 'Lugo', 'Barcelona', 'Madrid']
```

```
f = lambda x: len(x)
for s in cities:
    print(f(s))
```

```
# Explicación línea por línea:
# 1) f = lambda x: len(x) guarda la función anónima en una variable 'f', que se
    ↪ puede
#     invocar como cualquier otra función.
# 2) for s in cities recorre cada ciudad.
# 3) f(s) calcula la longitud de cada nombre y se imprime.
# Resultado esperado (longitudes de 'Valencia'=8, 'Lugo'=4, 'Barcelona'=9,
    ↪ 'Madrid'=6):
# 8
# 4
# 9
# 6
```

8

4

9
6

- Las expresiones lambda pueden formar parte de una lista.

```
[119]: pows = [lambda x: x ** 0,
               lambda x: x ** 1,
               lambda x: x ** 2,
               lambda x: x ** 3,
               lambda x: x ** 4]

for f in pows:
    print(f(2))

# Explicación línea por línea:
# 1) 'pows' es una lista de 5 funciones lambda, cada una elevando su argumento
#    a una
#    potencia distinta (0,1,2,3,4).
# 2) for f in pows recorre la lista; en cada iteración 'f' es una de esas
#    funciones.
# 3) f(2) invoca la lambda correspondiente con el argumento 2.
# 4) 2**0=1, 2**1=2, 2**2=4, 2**3=8, 2**4=16.
# Resultado esperado:
# 1
# 2
# 4
# 8
# 16
```

1
2
4
8
16

- Los diccionarios se pueden usar como **switch** de otros lenguajes; ideal para elegir entre varias opciones.

```
[120]: def switch_dict(operator, x, y):
        # crea un diccionario de opciones y selecciona 'operator'
        return {
            'add': lambda: x + y,
            'sub': lambda: x - y,
            'mul': lambda: x * y,
            'div': lambda: x / y,
        }.get(operator, lambda: None)() # retorna None si no encuentra 'operator'

print(switch_dict('add', 2, 2))
print(switch_dict('div', 10, 2))
```

```

print(switch_dict('mod', 17, 3))

# Explicación línea por línea:
# 1) Se construye un diccionario donde cada clave ('add', 'sub', 'mul', 'div')
    ↳ asocia a una
#     lambda SIN argumentos que ya "captura" (closure) los valores de x e y.
# 2) '.get(operator, lambda: None)' busca la clave 'operator' en el diccionario;
    ↳ si no la
#     encuentra, usa como valor por defecto una lambda que retorna None.
# 3) El '()' final INVOCA la lambda seleccionada (sea la real o la de por
    ↳ defecto).
# 4) switch_dict('add',2,2) -> selecciona la lambda de 'add' -> 2+2=4.
# 5) switch_dict('div',10,2) -> selecciona 'div' -> 10/2=5.0.
# 6) switch_dict('mod',17,3) -> 'mod' no existe en el diccionario -> usa la
    ↳ lambda por
#     defecto -> retorna None.
# Resultado esperado:
# 4
# 5.0
# None

```

4
5.0
None

3.8 8. Scopes (ámbitos)

- La localización de una asignación a una variable en el código determina desde dónde se puede acceder a ella.
- Regla LEGB: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-ins.

```

[121]: X = 10

def func():
    X = 20 # Local a la función. Es una variable diferente, no visible fuera
    ↳ de 'func'

    def func_int():
        X = 30
        print(X)

    func_int()

func()

# Explicación línea por línea:
# 1) X = 10 es una variable GLOBAL.

```

```

# 2) Dentro de 'func', 'X = 20' crea una variable LOCAL a func, distinta de la
    ↳ global (aunque
#     tenga el mismo nombre, es un objeto/espacio de memoria diferente).
# 3) Dentro de 'func_int' (anidada), 'X = 30' crea otra variable LOCAL a
    ↳ func_int (más interna
#     aún), distinta de la de func y de la global.
# 4) print(X) dentro de func_int imprime la X más cercana según la regla LEGB:
    ↳ la Local de
#     func_int, que es 30.
# 5) func() invoca a func, que a su vez invoca a func_int.
# Resultado esperado: 30

```

30

Regla LEGB con ejemplo de scopes Global y Local combinados:

```

[122]: X = 99  # Global (X y func)

def func(Y):  # Local (Y y Z)
    Z = X + Y
    return Z

print(func(1))

# Explicación línea por línea:
# 1) X = 99 es global.
# 2) func(Y) recibe un parámetro Y (local) y define Z (también local) como X+Y.
# 3) Dentro de la función, 'X' no está definida localmente, así que Python la
    ↳ busca en el
#     scope Global (regla LEGB), donde encuentra X=99.
# 4) Z = 99 + 1 = 100; se retorna ese valor.
# Resultado esperado: 100

```

100

```

[123]: X = 1

def func():
    X = 2  # X es una variable diferente (local)

func()
print(X)

# Explicación línea por línea:
# 1) X = 1 en el scope global.
# 2) Dentro de func, 'X = 2' crea una variable LOCAL nueva, que NO afecta a la
    ↳ X global
#     (son variables distintas que solo comparten el nombre).

```



```
# 3) func() se ejecuta pero no imprime nada ni modifica la X global.
# 4) print(X) fuera de la función muestra la X global, que sigue siendo 1.
# Resultado esperado: 1
```

1

- La sentencia `global` permite modificar una variable global desde dentro de una función.

```
[124]: X = 1

def func():
    global X
    X = 2

func()
print(X)

# Explicación línea por línea:
# 1) X = 1 en el scope global.
# 2) 'global X' dentro de la función le indica a Python que, al escribir X
    ↪ dentro de esta
#    función, se refiere a la variable GLOBAL, no a una nueva variable local.
# 3) X = 2 modifica entonces directamente la variable global.
# 4) func() se ejecuta, cambiando X a 2 en el scope global.
# 5) print(X) fuera de la función confirma el cambio.
# Resultado esperado: 2
```

2

- No hay necesidad de usar `global` para solo LEER variables globales; solo para modificarlas.

```
[125]: y, z = 1, 2

def todas_globales():
    global x
    x = y + z # Las tres variables son globales

todas_globales()
print(x)
print(y)
print(z)

# Explicación línea por línea:
# 1) y=1, z=2 son variables globales.
# 2) 'global x' declara que 'x', al asignarse dentro de la función, será GLOBAL
    ↪ (no local),
#    aunque 'x' todavía no existía antes de esta función.
# 3) x = y + z lee y y z (que ya son globales, no requieren la palabra 'global'
    ↪ para leerse)
```

```
# y crea x=3 como variable global.
# 4) Tras invocar la función, x ya existe en el scope global con valor 3.
# 5) Se imprimen las tres variables globales.
# Resultado esperado:
# 3
# 1
# 2
```

```
3
1
2
```

[126]:

```
X = 77

def f1():
    X = 88 # Enclosing scope (para f2)

    def f2():
        print(X)

    f2()

f1()

# Explicación línea por línea:
# 1) X = 77 en el scope global.
# 2) Dentro de f1, X = 88 crea una variable LOCAL a f1 (distinta de la global).
    ↳ Para la
# función anidada f2, esta X de f1 actúa como scope "Enclosing" (envolvente).
# 3) Dentro de f2, no hay X local, así que Python la busca según LEGB: primero
    ↳ Local (no existe
# en f2), luego Enclosing (la X=88 de f1) -> la encuentra ahí.
# 4) print(X) dentro de f2 imprime 88 (no 77, porque el Enclosing tiene
    ↳ prioridad sobre Global).
# 5) f1() invoca internamente a f2().
# Resultado esperado: 88
```

```
88
```

3.8.1 Closures

Las funciones pueden recordar su scope envolvente (enclosing), incluso si este ya no existe en el momento de la ejecución.

[127]:

```
def f1():
    X = 88 # Enclosing scope (para f2)

    def f2():
```

```

    print(X)

    return f2 # Devuelve f2, sin invocarla

accion = f1()
accion()

# Explicación línea por línea:
# 1) f1 define X=88 y una función interna f2 que usa esa X.
# 2) 'return f2' NO ejecuta f2; devuelve la función misma (el objeto función)
    ↪ como resultado
#     de f1. Este f2 "recuerda" el valor de X=88 de su scope envolvente, aunque
    ↪ f1 ya haya
#     terminado de ejecutarse: esto es un CLOSURE.
# 3) accion = f1() ejecuta f1 (que retorna f2) y guarda esa función en 'accion'.
# 4) accion() invoca finalmente f2, que imprime el X=88 que recordaba de f1, a
    ↪ pesar de que
#     f1 ya terminó su ejecución hace tiempo.
# Resultado esperado: 88

```

88

- Sentencia `nonlocal` para modificar variables que no son locales, pero tampoco globales.

```

[128]: def f1():
        contador = 10

        def f2():
            nonlocal contador
            contador += 1
            print(contador)

        return f2

accion = f1()
accion()
accion()
accion()

# Explicación línea por línea:
# 1) f1 define 'contador = 10' en su propio scope (enclosing para f2).
# 2) 'nonlocal contador' dentro de f2 indica que 'contador' NO es una variable
    ↪ local nueva
#     de f2, ni tampoco global: es la variable del scope envolvente (f1), y se
    ↪ podrá modificar
#     directamente (a diferencia de solo leerla).
# 3) contador += 1 incrementa esa variable compartida cada vez que se llama a
    ↪ f2.

```

```
# 4) accion = f1() crea UNA instancia del closure, con su propio 'contador'
↳ independiente.
# 5) Cada llamada a accion() incrementa el mismo 'contador' persistente entre
↳ llamadas
# (10->11, 11->12, 12->13), demostrando que el closure "recuerda" y puede
↳ modificar el
# estado entre invocaciones.
# Resultado esperado:
# 11
# 12
# 13
```

```
11
12
13
```

3.9 9. Ejercicios (enunciados)

1. Escribe una función que reciba una lista con números y devuelva una lista con los cuadrados de los números.
2. Escribe una función que reciba una cantidad indeterminada de números (sin recibir directamente un objeto complejo) y devuelva la suma de los mismos.
3. Escribe una función que reciba un string y devuelva el string al revés (ej: 'hola' -> 'aloh').
4. Escribe una función lambda que, igual que el ejercicio anterior, invierta un string.
5. Escribe una función que compruebe si un número se encuentra dentro de un rango específico.
6. Escribe una función que reciba un entero positivo y devuelva una lista con sus 5 primeros múltiplos. Si el parámetro no es válido (≤ 0), mostrar un mensaje de error y devolver una lista vacía.
7. Escribe una función que reciba una lista y compruebe si tiene elementos duplicados (True/False).
8. Escribe una función lambda equivalente al ejercicio anterior (comprobar duplicados).
9. Escribe una función que compruebe si un string dado es un palíndromo.

3.10 10. Soluciones propuestas

```
[129]: def cuadrados(numeros):
        return [n**2 for n in numeros]

print(cuadrados([1, 2, 3, 4, 5]))

# Explicación línea por línea (Ejercicio 1):
# 1) cuadrados(numeros) recibe una lista de números.
# 2) Usa una list comprehension para elevar al cuadrado (n**2) cada elemento de
↳ la lista.
# 3) Devuelve la nueva lista de cuadrados, sin modificar la lista original.
# Resultado esperado: [1, 4, 9, 16, 25]
```

```
[1, 4, 9, 16, 25]
```

```
[130]: def sumar_todos(*numeros):
        return sum(numeros)

print(sumar_todos(1, 2, 3))
print(sumar_todos(4, 5, 6, 7, 8))

# Explicación línea por línea (Ejercicio 2):
# 1) '*numeros' permite recibir una cantidad indeterminada de argumentos
    ↪ numéricos sueltos
#     (no una lista pasada directamente), empaquetados en una tupla llamada
    ↪ 'numeros'.
# 2) sum(numeros) suma todos los elementos de esa tupla.
# 3) sumar_todos(1,2,3) -> 1+2+3=6.
# 4) sumar_todos(4,5,6,7,8) -> 4+5+6+7+8=30.
# Resultado esperado:
# 6
# 30
```

6
30

```
[131]: def invertir_string(s):
        return s[::-1]

print(invertir_string('hola'))

# Explicación línea por línea (Ejercicio 3):
# 1) invertir_string(s) recibe un string.
# 2) 's[::-1]' es slicing con paso -1, que recorre el string de atrás hacia
    ↪ adelante,
#     produciendo el string invertido.
# 3) Se retorna ese string invertido.
# Resultado esperado: "aloh"
```

aloh

```
[132]: invertir_lambda = lambda s: s[::-1]
print(invertir_lambda('hola'))

# Explicación línea por línea (Ejercicio 4):
# 1) Misma lógica que el ejercicio anterior, pero expresada como una función
    ↪ lambda de una
#     sola línea (sin necesidad de 'def' ni 'return' explícito).
# 2) s[::-1] invierte el string recibido.
# Resultado esperado: "aloh"
```

aloh

```
[133]: def en_rango(numero, minimo, maximo):
        return minimo <= numero <= maximo

print(en_rango(5, 1, 10))
print(en_rango(15, 1, 10))

# Explicación línea por línea (Ejercicio 5):
# 1) en_rango recibe el número a comprobar y los límites del rango (minimo,
    ↪ maximo).
# 2) 'minimo <= numero <= maximo' es una comparación encadenada de Python:
    ↪ equivale a
#     (minimo <= numero) and (numero <= maximo).
# 3) en_rango(5,1,10) -> 1<=5<=10 -> True.
# 4) en_rango(15,1,10) -> 1<=15<=10 -> False (15 no es <=10).
# Resultado esperado:
# True
# False
```

True
False

```
[134]: def primeros_multiplos(n):
        if n <= 0:
            print("Error: el número debe ser positivo")
            return []
        return [n * i for i in range(1, 6)]

print(primeros_multiplos(3))
print(primeros_multiplos(-2))

# Explicación línea por línea (Ejercicio 6):
# 1) primeros_multiplos(n) primero valida que n sea positivo (n > 0).
# 2) Si n <= 0, imprime un mensaje de error y retorna una lista vacía de
    ↪ inmediato.
# 3) Si n es válido, construye una lista con los 5 primeros múltiplos de n
    ↪ usando una
#     comprehension: n*1, n*2, n*3, n*4, n*5.
# 4) primeros_multiplos(3) -> [3,6,9,12,15].
# 5) primeros_multiplos(-2) -> no es positivo -> imprime el error y retorna [].
# Resultado esperado:
# [3, 6, 9, 12, 15]
# Error: el número debe ser positivo
# []
```

[3, 6, 9, 12, 15]
Error: el número debe ser positivo
[]

```
[135]: def tiene_duplicados(lista):
        return len(lista) != len(set(lista))

print(tiene_duplicados([1, 2, 3, 4]))
print(tiene_duplicados([1, 2, 2, 4]))

# Explicación línea por línea (Ejercicio 7):
# 1) set(lista) elimina automáticamente los elementos duplicados (los conjuntos
    ↪no admiten
#     elementos repetidos).
# 2) Si len(lista) (con posibles duplicados) es distinto de len(set(lista))
    ↪(sin duplicados),
#     significa que SÍ había elementos repetidos.
# 3) tiene_duplicados([1,2,3,4]) -> longitudes 4 y 4 -> son iguales -> False
    ↪(no hay duplicados).
# 4) tiene_duplicados([1,2,2,4]) -> longitudes 4 y 3 -> son distintas -> True
    ↪(hay duplicados).
# Resultado esperado:
# False
# True
```

False

True

```
[136]: tiene_duplicados_lambda = lambda lista: len(lista) != len(set(lista))
print(tiene_duplicados_lambda([1, 2, 3, 4]))
print(tiene_duplicados_lambda([1, 2, 2, 4]))

# Explicación línea por línea (Ejercicio 8):
# 1) Misma lógica que el ejercicio anterior, pero como una expresión lambda de
    ↪una sola línea.
# 2) Compara la longitud original contra la longitud del conjunto (sin
    ↪duplicados).
# Resultado esperado:
# False
# True
```

False

True

```
[137]: def es_palindromo(s):
        s = s.lower().replace(" ", "")
        return s == s[::-1]

print(es_palindromo("reconocer"))
print(es_palindromo("python"))

# Explicación línea por línea (Ejercicio 9):
```

```
# 1) s.lower() convierte el string a minúsculas, para que la comparación no
↳distinga
# mayúsculas de minúsculas.
# 2) .replace(" ", "") elimina espacios, para que frases con espacios también
↳puedan
# evaluarse correctamente como palíndromos.
# 3) 's == s[::-1]' compara el string normalizado con su versión invertida; si
↳son iguales,
# es un palíndromo.
# 4) "reconocer" invertido es "reconocer" -> True.
# 5) "python" invertido es "nohtyp" -> distinto -> False.
# Resultado esperado:
# True
# False
```

True

False